Tutoriel GX Studio

Créer un casse-brique pour la GX4000, de A à Z

Sprites matériels, bruitages DMA, couleurs par rasters — cartouche CPR

GX Studio 0.5.1 — (c) 2026 ALONSO Stéphane [Fanou]



Le résultat : CASSE-BRIQUE GX sur GX4000

Ce tutoriel construit un vrai petit jeu avec GX Studio : une raquette et une balle en **sprites matériels**, soixante briques dessinées dans l'écran, des **bruitages** joués par le DMA de la console, un score, trois vies, et une couleur différente pour chaque rangée de briques grâce aux **interruptions raster**. Chaque étape se compile et se teste : on voit le jeu prendre forme au fur et à mesure. Le projet complet est fourni avec ce document (dossier CasseBrique). Le chapitre 12 refait le même jeu **en C** (dossier CasseBriqueC), pour ceux qui préfèrent ce langage ou veulent comparer.

Étape	Ce que l'on fait	Ce que l'on apprend
1	Créer le projet	cible GX4000, organisation d'un projet
2	Dessiner la raquette et la balle	éditeur de sprites, palettes, zoom des sprites
3	Créer les bruitages	éditeur de sons, écoute, canaux DMA
4	Écrire le programme	variables, boucle de jeu, bibliothèque
5	Les briques	dessiner dans la mémoire écran
6	La raquette et la balle	manette, sprites, rebonds, collisions
7	Score, vies, tableaux	nombres BCD, texte
8	Une couleur par rangée	interruptions raster de l'ASIC
9	Compiler, tester, déboguer	erreurs, points d'arrêt, Caprice32

10	Jouer sur la vraie console	copie de la cartouche sur la PicoGX
12	Le même jeu en C	SDCC, API gx.h, C et assembleur, débogage du C

Avant de commencer

GX Studio **0.5.1** doit être installé et réglé (voir la documentation de GX Studio, chapitre 1) : Java 17 ou plus, JavaFX, rasm et Caprice32 (avec le correctif GX Studio 0.5.1 pour voir les rasters correctement et déboguer dans le studio). Dans **Outils > Réglages**, les chemins de rasm et de cap32 doivent être renseignés. Quelques notions d'assembleur Z80 aident, mais chaque routine est expliquée.

Rappels sur l'écran utilisé : le **mode 1** affiche 320 x 200 pixels en 4 couleurs (encres 0 à 3). Les **sprites matériels** de la GX4000 se placent en pixels « mode 2 » (640 de large) : 1 pixel mode 1 = 2 pixels mode 2. Les lignes vont de 0 (en haut) à 199.

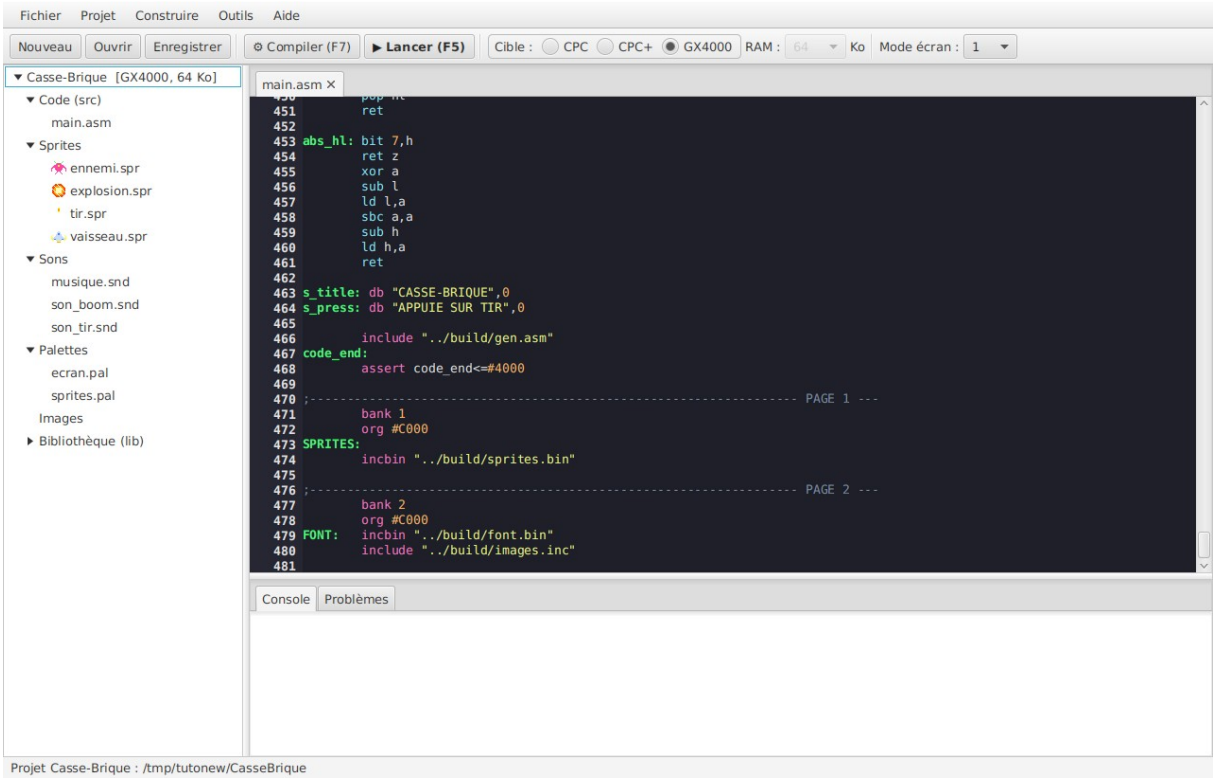
1. Créer le projet

Fichier > Nouveau projet. Donner le nom **Casse-Brique**, choisir le dossier parent, cocher **GX4000** (la mémoire passe à 64 Ko, la seule possible sur la console) et garder le **mode écran 1**.



La boîte « Nouveau projet »

Le studio crée le dossier du projet et y copie la bibliothèque GX4000 (lib/) et un petit jeu de départ (un vaisseau qui tire sur un ennemi). Il se lance déjà avec **F5** : c'est une bonne vérification que rasm et Caprice32 sont bien réglés. Nous allons remplacer ce jeu de départ par le casse-brique.



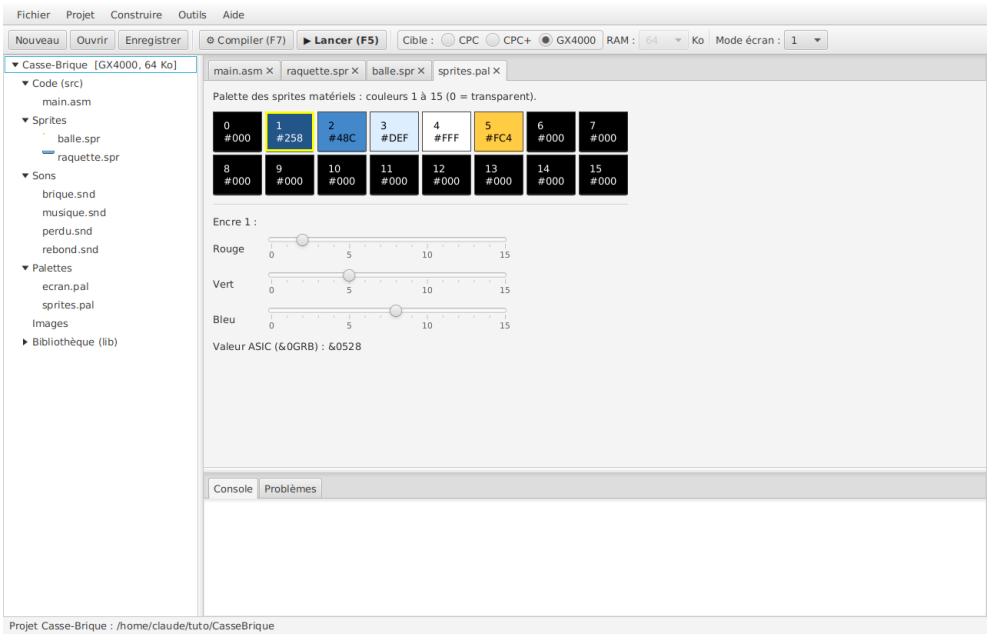
Le projet juste après sa création

Pour faire le ménage : dans l'arbre du projet, clic droit sur **vaisseau.spr**, **tir.spr**, **ennemi.spr**, **explosion.spr**, **son_tir.snd** et **son_boom.snd** puis **Supprimer**. On garde **musique.snd** pour l'écran titre et les deux palettes.

2. Dessiner les sprites

2.1 La palette des sprites

Ouvrir **Palettes > sprites.pal**. Les sprites matériels ont leurs propres couleurs 1 à 15 (la couleur 0 est toujours transparente). Cliquer sur une case puis régler le rouge, le vert et le bleu (0 à 15). Pour le casse-brique : 1 bleu foncé, 2 bleu moyen, 3 bleu très clair (la raquette), 4 blanc et 5 jaune (la balle).

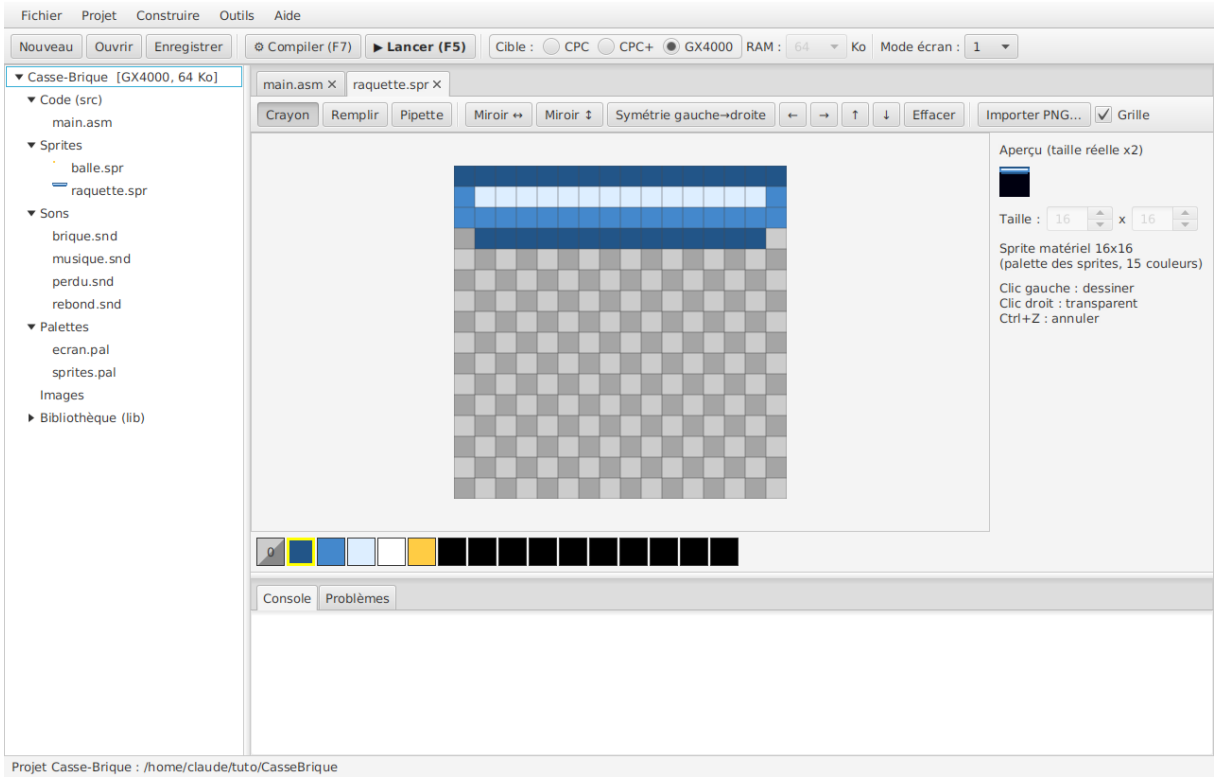


Palette des sprites : la valeur ASIC &0GRB est affichée sous les curseurs

Dans **ecran.pal**, seules les encres 0 à 3 servent en mode 1 : 0 = fond bleu nuit, 3 = blanc pour le texte et les briques (leur couleur sera changée par rangée à l'étape 8).

2.2 La raquette

Projet > Nouveau sprite…, nom **raquette**. Un sprite matériel fait toujours 16 x 16 pixels. La raquette n'utilise que les 4 premières lignes : on dessine une barre avec un liseré foncé (couleur 1), un cœur clair (3) et une ombre (2). Clic gauche pour dessiner avec la couleur choisie dans la barre du bas, clic droit pour effacer (transparent).

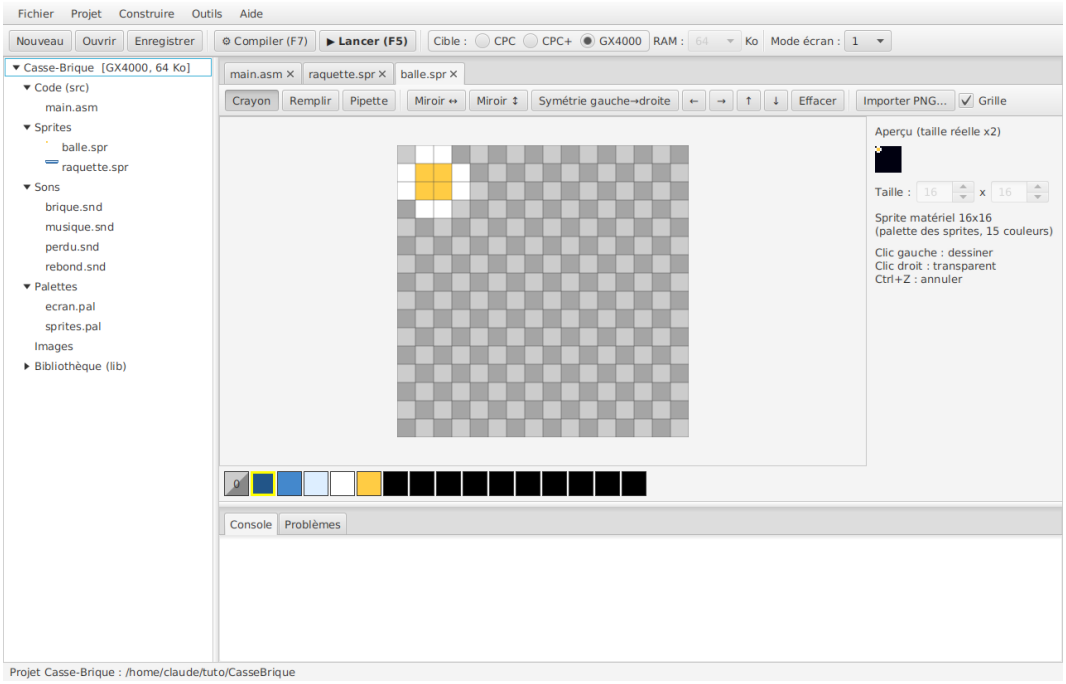


La raquette dans l'éditeur de sprites

Astuce : à l'écran, le programme affichera ce sprite avec un **zoom x4 en largeur** : 16 pixels deviennent 64 pixels mode 2, soit 32 pixels mode 1, une belle raquette, pour une seule image de 16 x 16.

2.3 La balle

Nouveau sprite **balle** : un petit rond de 4 x 4 pixels **dans le coin en haut à gauche** (couleurs 4 et 5). Le placer dans le coin simplifie les calculs : la position du sprite est celle de la balle.



La balle : 4 x 4 pixels, affichée avec un zoom x2 en largeur

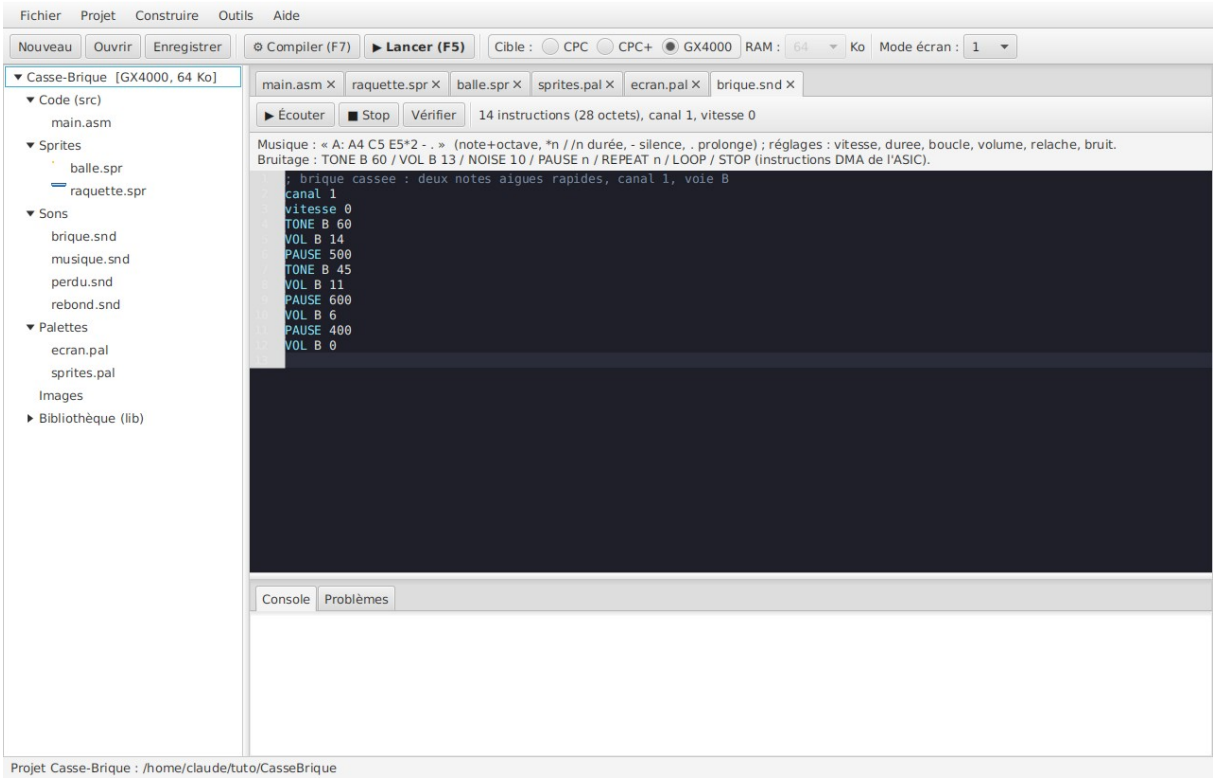
Les sprites sont de simples fichiers texte (un caractère par pixel), faciles à lire ou à modifier :

```
# GX Studio - sprite
size 16 16
1111111111111111
2333333333333332
2222222222222222
.11111111111111.
```

À la compilation, le studio range leurs images dans la page 1 de la cartouche et crée les numéros **F_BALLE** et **F_RAQUETTE** pour le programme (ordre alphabétique des fichiers).

3. Les bruitages

Projet > Nouveau son… pour chacun des trois bruitages. Un son est un petit texte : on peut l'écouter tout de suite avec ▶️ **Écouter**, sans lancer l'émulateur. Les instructions sont celles du **DMA de l'ASIC**, qui joue le son tout seul pendant que le jeu continue.



L'éditeur de sons : vérification et écoute

Fichier	Canal	Rôle
rebond.snd	1 (voie B)	la balle touche un mur ou la raquette
brique.snd	1 (voie B)	une brique est cassée (deux notes aiguës)
perdu.snd	2 (voie C, bruit)	la balle est perdue
musique.snd	0 (voies A et B)	la musique du titre (fournie)

```
; brique cassee : deux notes aigues rapides, canal 1, voie B
canal 1
vitesse 0
TONE B 60
VOL B 14
PAUSE 500
TONE B 45
VOL B 11
PAUSE 600
VOL B 6
PAUSE 400
VOL B 0
```

- `canal 1` : le son sera joué sur le canal DMA 1 ; le studio crée l'équate **BRIQUE_CH**.
- `vitesse 0` : une pause se compte en lignes d'écran (312 lignes = 1/50e de seconde) ; équate **BRIQUE_SPEED**.
- `TONE B 60` : hauteur de la voie B (plus le nombre est petit, plus c'est aigu) ; `VOL B 14` : volume (0-15) ; `PAUSE 500` : attendre 500 lignes.
- La liste est rangée en RAM par la bibliothèque ; son adresse est **DMA_RAM+BRIQUE_OFS**.

Le bruit de la balle perdue utilise la voie C en mode bruit (`NOISE 12` règle le « grain » du bruit) :

```
; balle perdue : note qui descend + bruit, canal 2, voie C
canal 2
vitesse 0
NOISE 12
VOL C 15
PAUSE 3000
NOISE 20
```

```
VOL C 12
PAUSE 3000
NOISE 28
VOL C 9
PAUSE 4000
NOISE 31
VOL C 5
PAUSE 4000
VOL C 0
```

3.1 Du fichier son au programme

Attention, c'est un point qui surprend souvent : **un son créé dans le studio n'est pas joué automatiquement**. À la compilation, le studio le transforme en liste DMA (dans build/gen.asm) et crée trois équates à son nom ; c'est ensuite le **programme** qui décide quand le jouer, en appelant `dma_play`. Un son qui n'est jamais appelé reste muet.

Équate créée pour brique.snd	Signification
BRIQUE_OFS	où se trouve la liste parmi tous les sons
BRIQUE_SPEED	la vitesse (ligne « vitesse » du fichier)
BRIQUE_CH	le canal DMA (ligne « canal » du fichier)

Le DMA de la console lit le son en RAM : au démarrage, `dma_init` recopie toutes les listes à l'adresse DMA_RAM (étape 2). Ensuite, à chaque fois qu'on veut entendre un son, trois lignes de chargement et un appel suffisent. Voici l'appel du bruit de brique, que l'on écrira à l'étape 7 bis, au moment précis où la brique est cassée :

```
        ld a,BRIQUE_CH                  ; A  = canal DMA (1)
        ld hl,DMA_RAM+BRIQUE_OFS        ; HL = adresse de la liste en RAM
        ld b,BRIQUE_SPEED               ; B  = vitesse
        call dma_play                   ; le son demarre, le jeu continue
```

Dans tout le jeu, il y a quatre appels de ce genre :

Son	Où il est lancé dans main.asm	Quand
musique	étape 2 (écran titre)	une fois, à l'affichage du titre
rebond	routine snd_rebond (étape 7), appelée par les rebonds	la balle touche un mur ou la raquette
brique	hit_bricks (étape 7 bis)	une brique disparaît
perdu	ball_lost (étape 7 bis)	la balle passe sous la raquette

- Un son se lance **une seule fois**, au moment de l'événement. L'appeler à chaque image le ferait redémarrer sans arrêt : on n'entendrait qu'un bourdonnement.
- Les 3 canaux DMA jouent en même temps, mais un nouveau `dma_play` sur un canal occupé coupe le son précédent. Ici la musique a le canal 0, rebond et brique le canal 1, perdu le canal 2 : un bruitage ne coupe jamais la musique.
- `dma_stop` (A = canal) arrête un son ; `dma_stop_all` arrête tout : c'est ce que fait l'étape 3 pour couper la musique du titre quand la partie commence.
- Un nouveau son se crée avec **Projet > Nouveau son…**, puis on ajoute ses trois lignes et `call dma_play` à l'endroit voulu. Pour vérifier son nom exact : F12 sur le nom, ou build/gen.asm.

4. Le programme : squelette et boucle de jeu

Ouvrir **Code > main.asm** et remplacer son contenu par celui du projet fourni (src/main.asm), étape

par étape, en lisant les explications. Le fichier est découpé en blocs « ETAPE n ».

4.1 Constantes et variables (étape 1)

Les constantes donnent un nom à chaque dimension du jeu : changer `BR_ROWS` ou `PAD_Y` suffit à modifier le terrain. Les variables sont réservées en RAM à partir de &8000 par la petite macro **VAR** ; les premières sont imposées par la bibliothèque.

```
;=== ETAPE 1 : les constantes du jeu ====================================
BR_COLS         equ 10                  ; 10 colonnes de briques
        ifndef BR_ROWS                  ; (peut etre change dans Projet > Proprietes :
BR_ROWS=2)
BR_ROWS         equ 6                   ; 6 rangees
        endif
BR_TOP          equ 32                  ; ligne de la premiere rangee
BR_H            equ 8                   ; hauteur d'une rangee (lignes)
BR_W            equ 8                   ; largeur d'une brique (octets = 32 pixels mode 1)
PAD_Y           equ 184                 ; ligne de la raquette
PAD_W           equ 64                  ; largeur de la raquette (pixels mode 2)
TOP_WALL        equ 14                  ; plafond (sous la ligne du score)
BALL_SPR        equ 0                   ; numeros des sprites materiels
PAD_SPR         equ 1
```

```
VAR bricks,BR_COLS*BR_ROWS              ; 1 = brique presente
VAR nbricks,1                           ; briques restantes
VAR pad_x,2                             ; raquette : X en pixels mode 2 (0-639)
VAR ball_x,2                            ; balle : X (pixels mode 2), Y (lignes)
VAR ball_y,1
VAR ball_dx,1                           ; vitesse (signee)
VAR ball_dy,1
VAR serve,1                             ; 1 = la balle attend sur la raquette
VAR lives,1
VAR score,2                             ; BCD (4 chiffres)
VAR speed,1                             ; vitesse verticale (2, puis 3)
VAR numbuf,8
```

4.2 L'organisation de la cartouche

Une cartouche est faite de pages de 16 Ko. La page 0 contient le code : elle commence obligatoirement par **lib/plus/boot.asm** (le démarrage de la console). La page 1 reçoit les images des sprites, la page 2 la police de caractères. Tout cela est déjà en place dans le fichier : il n'y a qu'à écrire le jeu entre les deux. Le démarrage inclut aussi l'**écran de présentation** de GX Studio (lib/common/splash.asm, environ 2,6 Ko de la page 0).

```
        bank 0
        org #0000
        include "../lib/plus/boot.asm"  ; doit etre au debut (vecteurs &0000 et &0038)
        include "../lib/plus/raster.asm"
        include "../lib/plus/sprites.asm"
        include "../lib/plus/dma.asm"
        include "../lib/common/input.asm"
        include "../lib/plus/text1.asm"
```

4.3 Démarrage et écran titre (étape 2)

`main` est appelé par la bibliothèque une fois la console prête, après les 3 secondes de l'écran de présentation de GX Studio. On recopie les sons en RAM (`dma_init`), on charge les palettes générées par le studio (`pal_screen`, `pal_sprites`), puis on affiche le titre, on lance la musique (`dma_play`, voir 3.1) et on attend un appui sur un bouton de tir.

```
;=== ETAPE 2 : demarrage et ecran titre ==================================
main:
        call dma_init                   ; listes de son recopiees en RAM
        ld hl,spr_have                  ; aucune image encore dans l'ASIC
        ld b,16
m_h:    ld (hl),#FF
        inc hl
        djnz m_h
```

```
        ld hl,pal_screen
        call set_palette4               ; encres 0-3 (assets/palettes/ecran.pal)
        ld hl,pal_sprites
        call set_spr_palette            ; couleurs 1-15 des sprites
title:
        call spr_hide_all
        ld hl,raster_idle               ; pas de couleurs par rangee sur le titre
        call raster_start
        call clear_screen
        ld a,3
        call set_ink
        ld bc,13*256+70
        ld de,s_title
        call text_at
        ld a,2
        call set_ink
        ld bc,13*256+120
        ld de,s_press
        call text_at
        ld a,MUSIQUE_CH                 ; musique (assets/sons/musique.snd)
        ld hl,DMA_RAM+MUSIQUE_OFS
        ld b,MUSIQUE_SPEED
        call dma_play
t_wait: call wait_frame
        call read_joy
        ld a,(joy_new)
        and (1<<J_FIRE1)|(1<<J_FIRE2)
        jr z,t_wait
```

4.4 La boucle de jeu (étapes 3 et 4)

Le cœur de tout jeu : une boucle exécutée 50 fois par seconde. `wait_frame` attend que l'écran ait fini d'être affiché (ligne 199) ; c'est le bon moment pour envoyer les nouvelles positions des sprites à l'ASIC (`spr_commit`) et charger les images qui ont changé (`spr_load`). Ensuite on lit les manettes et on fait avancer la raquette et la balle.

```
;=== ETAPE 3 : une nouvelle partie ============================================
        call dma_stop_all
        call clear_screen
        ld a,3
        ld (lives),a
        ld a,2
        ld (speed),a
        ld hl,0
        ld (score),hl
        ld hl,(640-PAD_W)/2
        ld (pad_x),hl
        call new_level
        ld hl,raster_bricks             ; couleurs par rangee (etape 8)
        call raster_start
        call show_score

;=== ETAPE 4 : la boucle de jeu (50 fois par seconde) ==========================
loop:
        call wait_frame                 ; ligne 199 : l'ecran visible est fini
        xor a
        call spr_commit                 ; positions calculees a l'image precedente
        call spr_load                   ; images de sprites qui ont change
        call read_joy
        call move_pad
        call move_ball
        ld a,(lives)
        or a
        jr nz,loop
        ; --- plus de vie : fin de partie
        call spr_hide_all
        ld a,3
        call set_ink
        ld bc,15*256+120
        ld de,s_over
        call text_at
        ld b,150
        call wait_frames
```

```
        jp title
```

5. Les briques

Les briques ne sont pas des sprites (il n'y en a que 16) : on les dessine directement dans la **mémoire écran**, qui commence en &C000. En mode 1, un octet contient 4 pixels ; une brique fait 8 octets de large (32 pixels) et 7 lignes de haut, la 8e ligne et le dernier pixel restant noirs pour séparer les briques. La table **line_addr**, générée par le studio, donne l'adresse de chaque ligne de l'écran (les lignes du CPC ne se suivent pas en mémoire : la ligne 1 est 2048 octets après la ligne 0).

- Le motif **#FF** allume les 4 pixels d'un octet avec l'encre 3 ; **0** les éteint (pour effacer une brique cassée).
- Le tableau **bricks** (60 octets) mémorise les briques présentes : 1 = présente, 0 = cassée. Une brique se trouve à l'indice rangée x 10 + colonne.

```
;=== ETAPE 5 : les briques ============================================
; new_level : remplit le tableau et dessine les 60 briques, balle sur la raquette
new_level:
        ld hl,bricks
        ld b,BR_COLS*BR_ROWS
nl_1:   ld (hl),1
        inc hl
        djnz nl_1
        ld a,BR_COLS*BR_ROWS
        ld (nbricks),a
        ld c,0                          ; C = rangee
nl_r:   ld b,0                          ; B = colonne
nl_c:   push bc
        ld a,#FF                        ; encre 3 sur 4 pixels
        call draw_brick
        pop bc
        inc b
        ld a,b
        cp BR_COLS
        jr nz,nl_c
        inc c
        ld a,c
        cp BR_ROWS
        jr nz,nl_r
        ld a,1                          ; la balle attend sur la raquette
        ld (serve),a
        ret

; draw_brick : B = colonne, C = rangee, A = motif (#FF = brique, 0 = effacer)
;   7 lignes de 8 octets ; le dernier pixel de chaque brique reste noir (joint)
draw_brick:
        ld e,a                          ; E = motif
        ld a,c                          ; ligne ecran = BR_TOP + rangee x 8
        add a,a
        add a,a
        add a,a
        add a,BR_TOP
        ld l,a
        ld h,0
        add hl,hl
        push de
        ld de,line_addr
        add hl,de                       ; HL -> adresse de la ligne
        pop de
        ld a,b                          ; colonne x 8 octets
        add a,a
        add a,a
        add a,a
        ld c,a
        ld d,7                          ; 7 lignes
dbr_l:   push hl
        ld a,(hl)
        inc hl
        ld h,(hl)
```

```
        add a,c
        ld l,a
        adc a,h
        sub l
        ld h,a                          ; HL = ecran
        ld b,BR_W-1
dbr_b:   ld (hl),e                       ; 7 octets pleins
        inc hl
        djnz dbr_b
        ld a,e
        and #EE                         ; 8e octet : 3 pixels sur 4
        ld (hl),a
        pop hl
        inc hl
        inc hl                          ; ligne suivante dans line_addr
        dec d
        jr nz,dbr_l
        ret
```

À ce stade, en ajoutant un simple `ret` au début de move_ball, on peut déjà compiler (**F5**) et voir le mur de briques et la raquette.

6. La raquette et la balle

6.1 La raquette

`read_joy` (bibliothèque) remplit **joy** avec l'état des deux manettes : un bit par direction et par bouton (J_LEFT, J_RIGHT, J_FIRE1…). La raquette avance de 8 pixels mode 2 par image et reste entre 0 et 640 - 64. `spr_set` place le sprite : numéro, X, Y, image (F_RAQUETTE) et zoom (**ZOOM_X4Y1** : x4 en largeur, x1 en hauteur).

```
;=== ETAPE 6 : la raquette ==========================================================
move_pad:
        ld hl,(pad_x)
        ld de,8                         ; 8 pixels mode 2 par image
        ld a,(joy)
        bit J_LEFT,a
        jr z,mp_r
        or a
        sbc hl,de
        jr nc,mp_r
        ld hl,0                         ; bord gauche
mp_r:   ld a,(joy)
        bit J_RIGHT,a
        jr z,mp_ok
        add hl,de
        ld de,640-PAD_W
        call cmp_hl_de
        jr c,mp_ok
        ex de,hl                        ; bord droit
mp_ok:  ld (pad_x),hl
        ld a,PAD_SPR
        ld de,PAD_Y
        ld bc,F_RAQUETTE*256+ZOOM_X4Y1  ; zoom x4 en largeur : 64 pixels mode 2
        jp spr_set
```

6.2 La balle et les rebonds

La balle a une position (ball_x, ball_y) et une vitesse (ball_dx, ball_dy), des nombres **signés** : -4 veut dire « 4 pixels vers la gauche à chaque image ». Avant le service, la balle suit la raquette ; un appui sur tir la lance vers le haut. Ensuite, à chaque image : on ajoute la vitesse à la position, et contre un mur on change le signe de la vitesse (rebond) en jouant le son **rebond** : la petite routine `snd_rebond` regroupe les quatre lignes de l'appel (voir 3.1), ce qui évite de les répéter à chaque rebond.

```
;=== ETAPE 7 : la balle =============================================================
move_ball:
        ld a,(serve)
```

```
        or a
        jr z,mb_fly
        ; --- service : la balle suit la raquette jusqu'a l'appui sur tir
        ld hl,(pad_x)
        ld de,PAD_W/2-4
        add hl,de
        ld (ball_x),hl
        ld a,PAD_Y-4
        ld (ball_y),a
        ld a,(joy_new)
        and (1<<J_FIRE1)|(1<<J_FIRE2)
        jr z,mb_draw
        xor a
        ld (serve),a
        ld a,4
        ld (ball_dx),a
        ld a,(speed)
        neg
        ld (ball_dy),a                  ; vers le haut
        jr mb_draw
mb_fly:
        ; --- deplacement horizontal et murs gauche / droit
        ld a,(ball_dx)
        ld e,a
        rla
        sbc a,a
        ld d,a                          ; DE = dx (signe)
        ld hl,(ball_x)
        add hl,de
        bit 7,h
        jr z,mb_x1
        ld hl,0                         ; mur gauche
        call bounce_x_pos
mb_x1:  ld de,640-8
        call cmp_hl_de
        jr c,mb_x2
        ex de,hl                        ; mur droit
        call bounce_x_neg
mb_x2:  ld (ball_x),hl
        ; --- deplacement vertical et plafond
        ld a,(ball_dy)
        ld b,a
        ld a,(ball_y)
        add a,b
        ld (ball_y),a
        cp TOP_WALL
        jr nc,mb_y1
        ld a,TOP_WALL
        ld (ball_y),a
        call bounce_y_pos
mb_y1:  call hit_bricks
        call hit_pad
        ld a,(ball_y)                   ; balle perdue ?
        cp 196
        jr c,mb_draw
        call ball_lost
mb_draw:
        ld a,BALL_SPR
        ld hl,(ball_x)
        ld de,(ball_y)
        ld d,0
        ld bc,F_BALLE*256+ZOOM_X2Y1
        jp spr_set
```

```
bounce_x_pos:                           ; dx devient positif
        ld a,(ball_dx)
        call abs_a
        ld (ball_dx),a
        jr snd_rebond
bounce_x_neg:                           ; dx devient negatif
        ld a,(ball_dx)
        call abs_a
        neg
        ld (ball_dx),a
        jr snd_rebond
```

```
bounce_y_pos:                           ; dy devient positif
        ld a,(ball_dy)
        call abs_a
        ld (ball_dy),a
snd_rebond:
        push hl
        ld a,REBOND_CH
        ld hl,DMA_RAM+REBOND_OFS
        ld b,REBOND_SPEED
        call dma_play
        pop hl
        ret
```

6.3 La collision avec la raquette

Si la balle descend et que son bas arrive au niveau de la raquette, on calcule où elle touche : la raquette est découpée en 8 zones. Au centre, la balle repart presque verticalement ; sur les bords, elle part en biais (table **pad_angle**). C'est ce qui permet au joueur de viser.

```
; hit_pad : la balle touche la raquette -> remonte, l'angle depend du point d'impact
hit_pad:
        ld a,(ball_dy)
        bit 7,a
        ret nz                          ; elle monte deja
        ld a,(ball_y)
        add a,4                         ; bas de la balle
        cp PAD_Y
        ret c
        cp PAD_Y+5
        ret nc
        ld hl,(ball_x)                  ; position relative : (balle+4) - raquette
        ld de,4
        add hl,de
        ld de,(pad_x)
        or a
        sbc hl,de
        ret c                           ; a gauche de la raquette
        ld de,PAD_W
        call cmp_hl_de
        ret nc                          ; a droite
        ld a,l                          ; 0..63 -> 8 zones
        rrca
        rrca
        rrca
        and 7
        ld e,a
        ld d,0
        ld hl,pad_angle
        add hl,de
        ld a,(hl)
        ld (ball_dx),a
        ld a,(speed)
        neg
        ld (ball_dy),a
        jr snd_rebond
pad_angle:  db -6,-4,-4,-2,2,4,4,6      ; vitesse horizontale selon la zone touchee
```

6.4 La collision avec les briques

On prend le **centre** de la balle. Sa ligne donne la rangée (une rangée = 8 lignes), son abscisse divisée par 64 donne la colonne (une brique = 64 pixels mode 2). Si la brique est présente : on l'efface du tableau et de l'écran, la balle repart dans l'autre sens vertical, on marque des points et on joue le son **brique**. Quand il n'y a plus de brique, un nouveau tableau commence, avec une balle plus rapide.

```
; hit_bricks : le centre de la balle est-il dans une brique ?
hit_bricks:
        ld a,(ball_y)
        add a,2                         ; centre vertical
        sub BR_TOP
        ret c
```

```
        cp BR_ROWS*BR_H
        ret nc
        rrca
        rrca
        rrca
        and 31
        ld c,a                          ; C = rangee
        ld hl,(ball_x)
        ld de,4
        add hl,de                       ; centre horizontal (0-639)
        ld b,0                          ; B = colonne = X / 64
        ld de,64
hb_d:   or a
        sbc hl,de
        jr c,hb_c
        inc b
        jr hb_d
hb_c:   ld a,b
        cp BR_COLS
        ret nc
        ld a,c                          ; indice = rangee x 10 + colonne
        add a,a
        ld e,a
        add a,a
        add a,a
        add a,e
        add a,b
        ld e,a
        ld d,0
        ld hl,bricks
        add hl,de
        ld a,(hl)
        or a
        ret z                           ; deja cassee
        ld (hl),0
        push bc
        xor a
        call draw_brick                 ; efface la brique
        pop bc
        ld a,(ball_dy)                  ; rebond vertical
        neg
        ld (ball_dy),a
        ld a,BR_ROWS                    ; points : 10 x (6 - rangee)
        sub c
        call add_points
        ld a,BRIQUE_CH
        ld hl,DMA_RAM+BRIQUE_OFS
        ld b,BRIQUE_SPEED
        call dma_play
        ld hl,nbricks
        dec (hl)
        ret nz
        ; --- tableau termine : on recommence, un peu plus vite
        ld a,3
        ld (speed),a
        jp new_level

; ball_lost : une vie de moins, la balle revient sur la raquette
ball_lost:
        ld a,PERDU_CH
        ld hl,DMA_RAM+PERDU_OFS
        ld b,PERDU_SPEED
        call dma_play
        ld hl,lives
        dec (hl)
        ld a,1
        ld (serve),a
        jp show_score
```

7. Score et vies

Le score est gardé en **BCD** : chaque octet contient deux chiffres décimaux. L'instruction `daa`

corrige l'addition pour rester en décimal, ce qui évite toute division au moment de l'affichage : chaque demi-octet est directement un chiffre. `text_at` (bibliothèque) écrit une chaîne à une colonne (0-39) et une ligne (0-199) données.

```
;=== ETAPE 7 bis : score et petites routines ==================================
; add_points : A = nombre de dizaines a ajouter (score BCD)
add_points:
        add a,a                         ; A x 10 en BCD : chiffre des dizaines
        add a,a
        add a,a
        add a,a
        ld b,a
        ld a,(score)
        add a,b
        daa
        ld (score),a
        ld a,(score+1)
        adc a,0
        daa
        ld (score+1),a
        jp show_score

show_score:
        ld a,3
        call set_ink
        ld bc,0*256+0
        ld de,s_score
        call text_at
        ld hl,numbuf
        ld a,(score+1)
        call bcd_out
        ld a,(score)
        call bcd_out
        ld (hl),0
        ld bc,6*256+0
        ld de,numbuf
        call text_at
        ld bc,32*256+0
        ld de,s_lives
        call text_at
        ld a,(lives)
        add a,'0'
        ld (numbuf),a
        xor a
        ld (numbuf+1),a
        ld bc,37*256+0
        ld de,numbuf
        jp text_at

bcd_out:
        ld c,a
        rrca
        rrca
        rrca
```

8. Une couleur par rangée : les rasters

C'est la touche « GX4000 » du jeu. Le mode 1 n'a que 4 encres, et pourtant chaque rangée a sa couleur : toutes les briques sont dessinées avec l'encre 3, et l'ASIC **change la couleur de l'encre 3 pendant l'affichage**, juste avant chaque rangée. Le registre PRI de l'ASIC déclenche une interruption à une ligne choisie ; la bibliothèque (lib/plus/raster.asm) enchaîne ces interruptions d'après une **table** : pour chaque entrée, un type, une donnée et la ligne de l'interruption suivante.

- **type 0** : nouvelle couleur de l'encre 3, au format de l'ASIC **&0GRB** (vert, rouge, bleu de 0 à F). Exemple : &00F2 = rouge, &0F40 = vert.
- **type 2** : fin de la zone visible (ligne 199), c'est ce qui réveille wait_frame.
- **type 3** : « armement » dans le bord bas (ligne 304) : la table recommence à l'image suivante.
- La première interruption est placée deux lignes avant la première rangée : le processeur met

un petit moment à réagir, et ces lignes sont vides.

```
;=== ETAPE 8 : une couleur par rangee de briques (interruptions raster) =======
; Les briques sont toutes dessinees avec l'encre 3. A chaque rangee, une
; interruption raster change la couleur de l'encre 3 ; apres la derniere
; rangee, l'encre 3 redevient blanche (pour le score en haut de l'ecran).
; Format : premiere ligne, puis entrees (type, donnee, ligne suivante).
raster_bricks:
        db BR_TOP-2                             ; un peu avant la brique (temps de reaction)
        db 0 : dw #00F2 : db BR_TOP-1+1*BR_H    ; type 0 = couleur de l'encre 3 : &0GRB
(rouge)
        db 0 : dw #08F0 : db BR_TOP-1+2*BR_H    ; orange
        db 0 : dw #0FF0 : db BR_TOP-1+3*BR_H    ; jaune
        db 0 : dw #0F40 : db BR_TOP-1+4*BR_H    ; vert
        db 0 : dw #0F0F : db BR_TOP-1+5*BR_H    ; cyan
        db 0 : dw #044F : db BR_TOP-1+6*BR_H    ; bleu
        db 0 : dw #0FFF : db 199                ; retour au blanc (score)
        db 2 : dw 0 : db ARM_LINE               ; fin d'image (ligne 199)
        db 3 : dw 0 : db 0                      ; armement (ligne 304) : on recommence
```

La table est activée une seule fois au début de la partie par `ld hl,raster_bricks` puis `call raster_start` (étape 3). Sur l'écran titre, `raster_idle` (table minimale de la bibliothèque) la remplace.



Six couleurs avec une seule encre : le travail des interruptions raster

9. Compiler, tester, déboguer

9.1 Compiler

F7 compile : le studio convertit d'abord les sprites, palettes et sons (fichiers du dossier build/), puis lance rasm. En cas d'erreur, l'onglet **Problèmes** liste les messages ; un double-clic ouvre le fichier à la bonne ligne. Les erreurs les plus fréquentes :

Message de rasm	Cause probable
keyword [XXX] not found	étiquette mal orthographiée, ou son/sprite renommé (F_XXX,

	XXX_OFS…)
Duplicate label [XXX]	étiquette déjà utilisée, souvent par la bibliothèque : changer de nom (ex. db_l existe dans dma.asm)
relative offset … too far	un jr trop loin : remplacer par jp
assert … code_end<=#4000	le code dépasse 16 Ko : déplacer des données dans une autre page

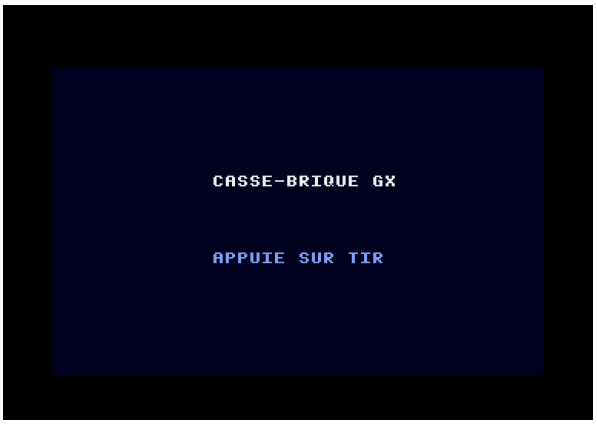
9.2 Lancer

F5 compile puis lance Caprice32 en 6128+ avec la cartouche. Comme tout jeu construit avec la bibliothèque du studio, le casse-brique commence par l'**écran de présentation** de GX Studio : pendant 3 secondes, le logo FANOU 1972 et « Jeux développé avec GXStudio By Fanou », puis l'écran titre du jeu. Rien n'est à programmer : c'est le démarrage de la bibliothèque (lib/plus/boot.asm) qui l'affiche.



L'écran de présentation de GX Studio (3 secondes au démarrage)

Dans l'émulateur, la manette se règle dans son menu (F1) ; on peut aussi jouer au clavier sur CPC+ (flèches et Espace), mais pas sur la vraie GX4000, qui n'a pas de clavier.

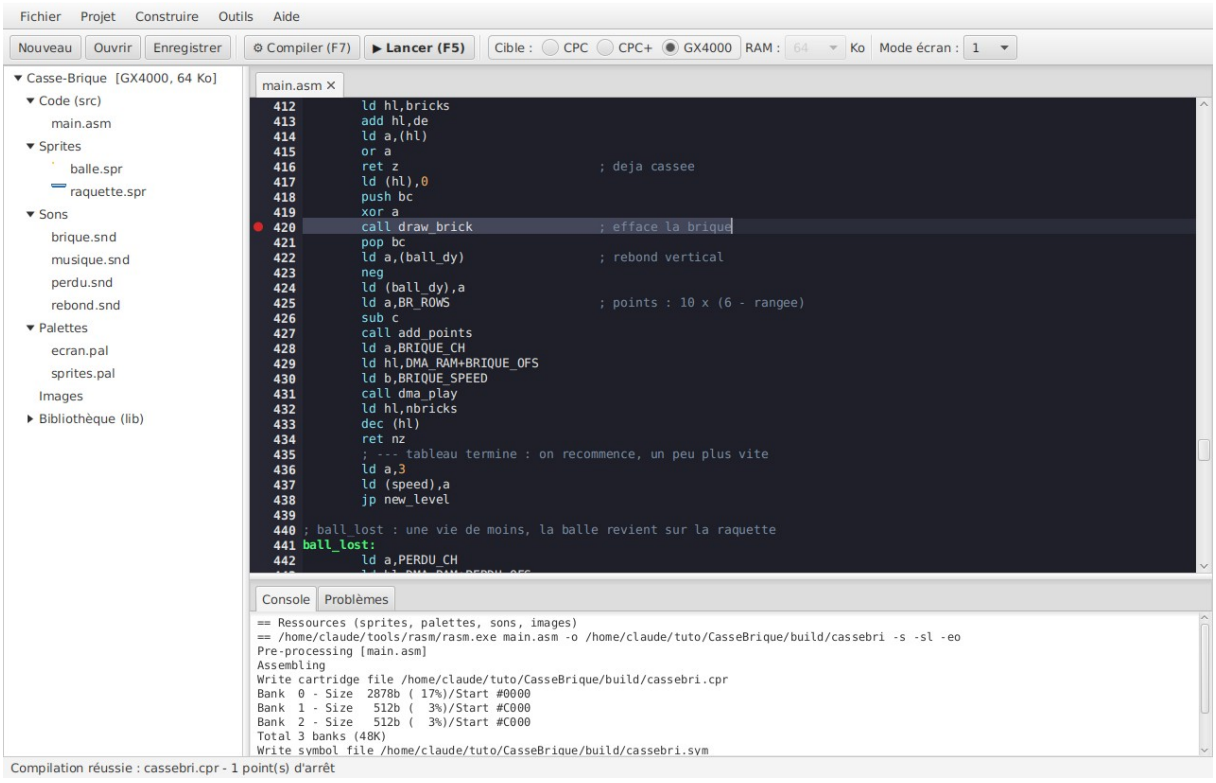




L'écran titre et une partie en cours dans Caprice32

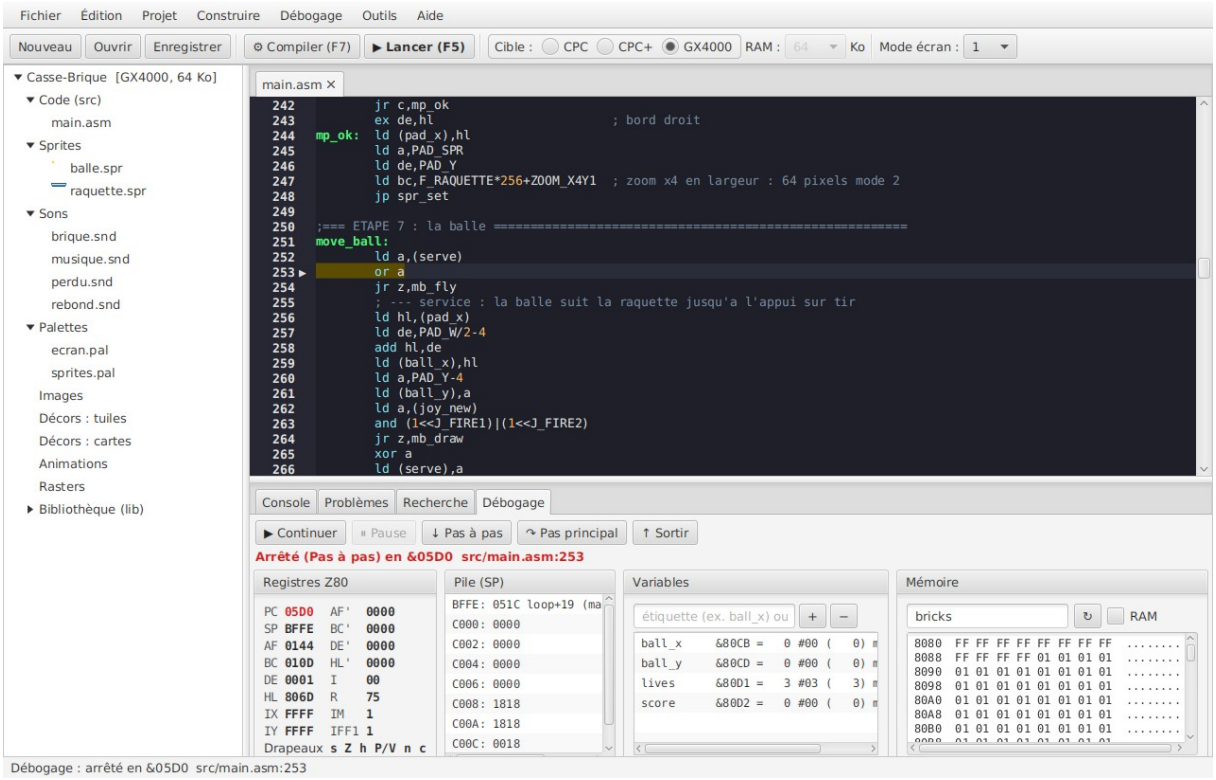
9.3 Poser un point d'arrêt

Pour comprendre ce qui se passe quand une brique est cassée : cliquer dans la marge à gauche de la ligne `call draw_brick ; efface la brique` (ou F9). Un rond rouge apparaît. **F5** : la compilation indique l'adresse du point d'arrêt, et dès que la balle touche une brique, le jeu s'arrête : le studio **surligne la ligne** dans l'éditeur (flèche jaune) et l'onglet **Débogage** montre les registres (B = colonne, C = rangée de la brique touchée ; en rouge ce qui a changé), la pile, la mémoire.



Un point d'arrêt sur l'effacement d'une brique

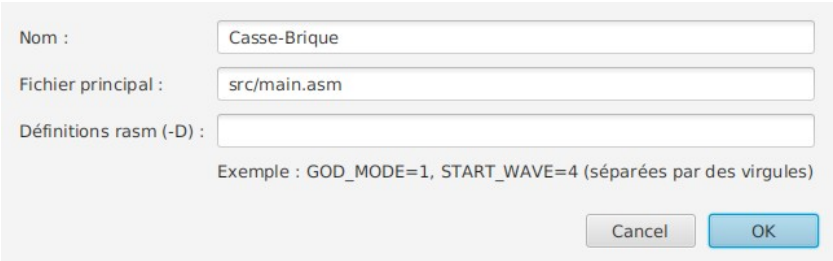
- **F11** avance d'une instruction (et entre dans draw_brick), **F10** exécute un CALL d'un coup, **Maj+F11** termine la routine, **F8** relance le jeu, **F6** l'arrête là où il en est.
- Les points d'arrêt se posent et s'enlèvent **pendant que le jeu tourne**.
- Dans **Variables**, taper `ball_x`, `ball_y`, `lives`, `score` (ou Ctrl+Maj+W sur un nom dans le code) : leurs valeurs s'affichent à chaque arrêt. Dans **Mémoire**, `bricks` montre le tableau des 60 briques (1 = présente).
- Il faut le Caprice32 corrigé de GX Studio 0.5.1 (caprice32/LISEZMOI.txt) ; avec un autre Caprice32, ses propres outils s'ouvrent au point d'arrêt (Maj+F2).



Pas à pas dans move_ball : ligne courante, registres, variables de la balle, tableau des briques

9.4 Options de mise au point

Projet > Propriétés permet de passer des définitions à rasm, sans toucher au code. Par exemple `BR_ROWS=2` pour tester rapidement la fin d'un tableau avec seulement deux rangées : dans le code, la définition de BR_ROWS est entourée par `ifndef BR_ROWS` … `endif`, elle n'est donc utilisée que si rien n'est défini dans les propriétés. Penser à vider ce champ ensuite.



Propriétés du projet : définitions passées à rasm (-D)

10. Sur la vraie GX4000

La cartouche est dans le dossier **build/** du projet : **cassebri.cpr** (nom court du projet, 8 lettres). Il suffit de la copier sur la carte de la **PicoGX** (ou de tout autre cartouche flashable au format CPR) et de démarrer la console. La même cartouche fonctionne sur un 464+ ou un 6128+.

11. Pour aller plus loin

- **Briques solides** : une valeur 2 dans le tableau `bricks` = il faut deux coups ; la dessiner avec l'encre 2.
- **Bonus qui tombent** : un troisième sprite (par exemple une capsule) lâché par certaines briques ; la raquette l'attrape (même test que la balle).
- **Raquette plus large** : un deuxième sprite raquette collé au premier, ou le zoom x4 en hauteur pour un effet de relief.

- **Musique de jeu** : un second fichier .snd joué en boucle sur le canal 0 pendant la partie (les bruitages utilisent les canaux 1 et 2).
- **Niveaux** : une table de motifs (quelles briques sont présentes) lue par new_level.
- **Mesurer** : **Débogage > Mesurer les performances** (Ctrl+F6) montre, sur 5 secondes de jeu, les routines qui prennent le plus de temps et le temps libre par image (documentation de GX Studio, 4.2).
- **Version CPC** : créer un projet CPC, la bibliothèque lib/cpc remplace les sprites matériels par des sprites logiciels (sw_set / sw_update) et le DMA par un lecteur de son qui lit les mêmes fichiers .snd.

12. Le même jeu en C

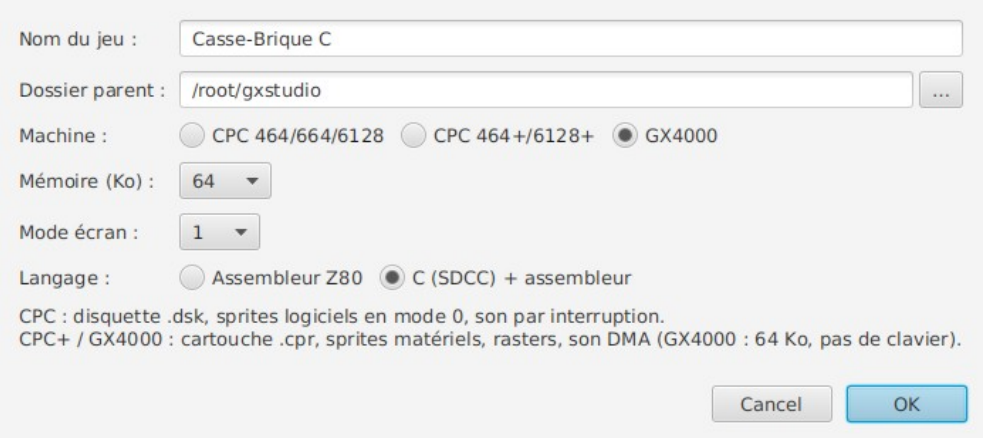
Depuis la version 0.5, GX Studio permet d'écrire un jeu en **C**, compilé pour le Z80 par **SDCC**, et de le mélanger avec de l'assembleur. Ce chapitre reprend le casse-brique, étape par étape, en C : les sprites, les sons et la palette sont **exactement les mêmes** (étapes 2 et 3) ; seul le programme change. Le projet complet est dans le dossier **CasseBriqueC** (aussi dans exemples/ de GX Studio). Le détail du C dans le studio (API, convention d'appel, mémoire) est au chapitre 6 de la documentation de GX Studio.

	Assembleur (chapitres 4 à 8)	C (ce chapitre)
Programme	src/main.asm (570 lignes)	src/main.c (230 lignes) ; main.asm ne fait plus qu'assembler la cartouche
Appels	ld a,… / ld hl,… / call dma_play	GX_SOUND(BRIQUE) ; gx_spr_set(…) ; gx_text(…)
Score	BCD et daa	un simple entier uint16_t, affiché par gx_text_num
Taille du code	2,9 Ko	4,3 Ko (le jeu tient toujours largement dans la page 0)
Vitesse		largement suffisante : la boucle de jeu tient dans une image (50 par seconde)

12.1 Installer SDCC et créer le projet

```
sudo apt install sdcc
```

Si `sdcc` n'est pas dans le PATH, indiquer son chemin dans **Outils > Réglages**. Puis **Fichier > Nouveau projet** : nom **Casse-Brique C**, **GX4000**, mode écran **1**, et langage **C (SDCC) + assembleur**.



Le choix du langage dans « Nouveau projet »

Le projet ressemble au projet en assembleur, avec deux différences : **src/main.c** contient le jeu de

départ (en C), et **lib/c** contient la liaison entre le C et la bibliothèque (gx.h, gx_c.asm, gx.c). Pour les ressources, refaire les étapes 2 et 3 (ou recopier le dossier assets/ du projet CasseBrique) : raquette.spr, balle.spr, les deux palettes, rebond.snd, brique.snd, perdu.snd et musique.snd.

src/main.asm est déjà prêt et n'a pas besoin d'être modifié : il réserve les variables de la bibliothèque et celles du C (`VAR c_ram,C_RAM_SIZE`), inclut la bibliothèque, la liaison et le code C traduit, puis appelle `main()` :

```
main:
        call c_init                     ; variables du C
        call _main                      ; main() de src/main.c
        jr main

        include "../build/gen.asm"      ; ressources (assets/)
        include "../build/c_code.asm"   ; le C, traduit pour rasm
```

12.2 Constantes et variables (étape 1)

Les `equ` deviennent des `#define`, les `VAR` des variables `static`. On choisit le plus petit type possible : `uint8_t` (0 à 255) pour presque tout, `int16_t` pour les abscisses (0 à 639), `int8_t` (signé) pour les vitesses. Les constantes des ressources (F_RAQUETTE, BRIQUE_CH…) viennent de **build/gen.h**, inclus par gx.h.

```
/*=== ETAPE 1 : les constantes du jeu ====================================*/
#define BR_COLS   10          /* 10 colonnes de briques */
#ifndef BR_ROWS               /* (peut etre change dans Projet > Proprietes : BR_ROWS=2) */
#define BR_ROWS   6           /* 6 rangees */
#endif
#define BR_TOP    32          /* ligne de la premiere rangee */
#define BR_H      8           /* hauteur d'une rangee (lignes) */
#define BR_W      8           /* largeur d'une brique (octets = 32 pixels mode 1) */
#define PAD_Y     184         /* ligne de la raquette */
#define PAD_W     64          /* largeur de la raquette (pixels mode 2) */
#define TOP_WALL  14          /* plafond (sous la ligne du score) */
#define BALL_SPR  0           /* numeros des sprites materiels */
#define PAD_SPR   1

/*=== ETAPE 1 : les variables ============================================*/
static uint8_t  bricks[BR_COLS * BR_ROWS];   /* 1 = brique presente */
static uint8_t  nbricks;                     /* briques restantes */
static int16_t  pad_x;                       /* raquette : X en pixels mode 2 (0-639) */
static int16_t  ball_x;                      /* balle : X (pixels mode 2), Y (lignes) */
static uint8_t  ball_y;
static int8_t   ball_dx, ball_dy;            /* vitesse */
static uint8_t  serve;                       /* 1 = la balle attend sur la raquette */
static uint8_t  lives;
static uint16_t score;
static uint8_t  speed;                       /* vitesse verticale (2, puis 3) */
```

12.3 Démarrage, titre et boucle de jeu (étapes 2 à 4)

`main()` suit le même plan que le programme en assembleur : initialisation, palettes, puis une boucle sans fin titre → partie. Chaque routine de la bibliothèque a sa fonction **gx_…** : `gx_wait_frame()` pour `wait_frame`, `gx_spr_update()` pour `spr_commit` + `spr_load`, `gx_read_joy()` pour `read_joy`. Après `gx_read_joy()`, les variables **joy** et **joy_new** sont lisibles directement en C.

```
/*=== ETAPE 2 : l'ecran titre ============================================*/
static void title(void)
{
    gx_spr_hide_all();
    gx_raster_start(raster_idle);                    /* pas de couleurs par rangee */
    gx_clear_screen();
    gx_ink(3);
    gx_text(13, 70, "CASSE-BRIQUE GX");
    gx_ink(2);
    gx_text(13, 120, "APPUIE SUR TIR");
    GX_SOUND(MUSIQUE);                               /* assets/sons/musique.snd */
    do {
```

```
        gx_wait_frame();
        gx_read_joy();
    } while (!(joy_new & JOY_FIRE));
}
```

```
/*=== ETAPES 2 a 4 : demarrage, partie, boucle de jeu ======================*/
void main(void)
{
    gx_init();                                      /* son (DMA), sprites */
    gx_palette(pal_screen);                         /* encres 0-3
(assets/palettes/ecran.pal) */
    gx_sprite_palette(pal_sprites);                 /* couleurs 1-15 des sprites */
    for (;;) {
        title();
        /*--- ETAPE 3 : une nouvelle partie */
        gx_sound_stop_all();
        gx_clear_screen();
        lives = 3;
        speed = 2;
        score = 0;
        pad_x = (640 - PAD_W) / 2;
        new_level();
        gx_raster_start(raster_bricks);             /* couleurs par rangee (etape 8) */
        show_score();
        /*--- ETAPE 4 : la boucle de jeu (50 fois par seconde) */
        while (lives) {
            gx_wait_frame();                        /* ligne 199 : l'ecran visible est fini
*/
            gx_spr_update();                        /* positions calculees a l'image
precedente */
            gx_read_joy();
            move_pad();
            move_ball();
        }
        /*--- plus de vie : fin de partie */
        gx_spr_hide_all();
        gx_ink(3);
        gx_text(15, 120, "GAME OVER");
        gx_wait_frames(150);
    }
}
```

Les sons se lancent avec **GX_SOUND(NOM)** : la macro utilise les trois constantes créées par le studio pour chaque fichier son (NOM_CH, NOM_OFS, NOM_SPEED), exactement comme les trois `ld` avant `call dma_play` (paragraphe 3.1). Même règle qu'en assembleur : on lance un son une seule fois, au moment de l'événement.

12.4 Les briques (étape 5)

`GX_LINE(y)` donne l'adresse de la ligne y de l'écran (la table line_addr générée par le studio) : un pointeur `uint8_t *` permet d'écrire les octets de la brique directement dans la mémoire écran.

```
/*=== ETAPE 5 : les briques ================================================*/
/* draw_brick : 7 lignes de 8 octets ; le dernier pixel reste noir (joint).
 * pattern = 0xFF (encre 3 sur 4 pixels) pour dessiner, 0 pour effacer. */
static void draw_brick(uint8_t col, uint8_t row, uint8_t pattern)
{
    uint8_t y = BR_TOP + row * BR_H, i, j;
    for (i = 0; i < BR_H - 1; i++) {
        uint8_t *p = GX_LINE(y + i) + col * BR_W;    /* adresse de l'octet a l'ecran */
        for (j = 0; j < BR_W - 1; j++) *p++ = pattern;
        *p = pattern & 0xEE;                         /* 8e octet : 3 pixels sur 4 */
    }
}

/* new_level : remplit le tableau et dessine les 60 briques */
static void new_level(void)
{
    uint8_t c, r;
    for (r = 0; r < BR_ROWS; r++)
        for (c = 0; c < BR_COLS; c++) {
            bricks[r * BR_COLS + c] = 1;
```

```
            draw_brick(c, r, 0xFF);
        }
    nbricks = BR_COLS * BR_ROWS;
    serve = 1;                                  /* la balle attend sur la raquette */
}
```

12.5 La raquette et la balle (étapes 6 et 7)

Les tests de la manette sont de simples `if (joy & JOY_LEFT)`. `gx_spr_set` prend le numéro du sprite, X (pixels mode 2), Y, l'image et le zoom.

```
/*=== ETAPE 6 : la raquette ======================================================*/
static void move_pad(void)
{
    if (joy & JOY_LEFT) {
        pad_x -= 8;                             /* 8 pixels mode 2 par image */
        if (pad_x < 0) pad_x = 0;
    }
    if (joy & JOY_RIGHT) {
        pad_x += 8;
        if (pad_x > 640 - PAD_W) pad_x = 640 - PAD_W;
    }
    gx_spr_set(PAD_SPR, pad_x, PAD_Y, F_RAQUETTE, ZOOM_X4Y1);  /* x4 en largeur : 64 pixels
*/
}
```

La balle : en C, les nombres signés et les comparaisons rendent les rebonds très lisibles ; `abs8` force le signe de la vitesse après un rebond.

```
static void move_ball(void)
{
    if (serve) {                                /* service : la balle suit la raquette
*/
        ball_x = pad_x + PAD_W / 2 - 4;
        ball_y = (uint8_t)(PAD_Y - 4);
        if (joy_new & JOY_FIRE) {
            serve = 0;
            ball_dx = 4;
            ball_dy = -(int8_t)speed;           /* vers le haut */
        }
    } else {
        ball_x += ball_dx;                      /* deplacement horizontal, murs */
        if (ball_x < 0) { ball_x = 0; ball_dx = abs8(ball_dx); rebound(); }
        if (ball_x >= 640 - 8) { ball_x = 640 - 8; ball_dx = -abs8(ball_dx); rebound(); }
        ball_y += ball_dy;                      /* deplacement vertical, plafond */
        if (ball_y < TOP_WALL) { ball_y = TOP_WALL; ball_dy = abs8(ball_dy); rebound(); }
        hit_bricks();
        hit_pad();
        if (ball_y >= 196) ball_lost();         /* balle perdue */
    }
    gx_spr_set(BALL_SPR, ball_x, ball_y, F_BALLE, ZOOM_X2Y1);
}
```

La collision avec la raquette utilise la même table **pad_angle** de 8 zones ; celle avec les briques le même calcul de rangée (division par 8 = `>> 3`) et de colonne (`>> 6`). On évite les vraies divisions : un décalage ne coûte que quelques instructions au Z80.

```
/* hit_pad : la balle touche la raquette -> remonte ; l'angle depend du point d'impact */
static const int8_t pad_angle[8] = { -6, -4, -4, -2, 2, 4, 4, 6 };
static void hit_pad(void)
{
    uint8_t bottom;
    int16_t rel;
    if (ball_dy < 0) return;                    /* elle monte deja */
    bottom = ball_y + 4;
    if (bottom < PAD_Y || bottom >= PAD_Y + 5) return;
    rel = ball_x + 4 - pad_x;                   /* position relative a la raquette */
    if (rel < 0 || rel >= PAD_W) return;
    ball_dx = pad_angle[rel >> 3];              /* 8 zones */
    ball_dy = -(int8_t)speed;
    rebound();
}
```

```
/* hit_bricks : le centre de la balle est-il dans une brique ? */
static void hit_bricks(void)
{
    uint8_t cy = ball_y + 2, row, col, k;
    if (cy < BR_TOP || cy >= BR_TOP + BR_ROWS * BR_H) return;
    row = (cy - BR_TOP) >> 3;
    col = (uint8_t)((ball_x + 4) >> 6);             /* 64 pixels mode 2 par brique */
    if (col >= BR_COLS) return;
    k = row * BR_COLS + col;
    if (!bricks[k]) return;                         /* deja cassee */
    bricks[k] = 0;
    draw_brick(col, row, 0);                        /* efface la brique */
    ball_dy = -ball_dy;                             /* rebond vertical */
    score += 10 * (BR_ROWS - row);                  /* points : 10 x (6 - rangee) */
    show_score();
    GX_SOUND(BRIQUE);
    if (--nbricks == 0) {                           /* tableau termine : un peu plus vite
*/
        speed = 3;
        new_level();
    }
```

12.6 Score et vies (étape 7 bis)

Plus besoin de BCD : le score est un entier `uint16_t`, et **gx_text_num** l'affiche sur 4 chiffres (elle est écrite en C dans lib/c/gx.c).

```
/*=== ETAPE 7 bis : score et vies ========================================*/
static void show_score(void)
{
    gx_ink(3);
    gx_text(0, 0, "SCORE");
    gx_text_num(6, 0, score, 4);
    gx_text(32, 0, "VIES");
    gx_text_num(37, 0, lives, 1);
}
```

12.7 Une couleur par rangée (étape 8)

La table raster devient un tableau `const` (il reste dans la cartouche). Les macros de gx.h écrivent les 4 octets de chaque entrée : **RASTER_INK_ENTRY(couleur, ligne suivante)** pour changer l'encre 3, **RASTER_END_ENTRY** pour la fin d'image et l'armement. La table s'active avec `gx_raster_start(raster_bricks)` au début de la partie, et `gx_raster_start(raster_idle)` sur l'écran titre.

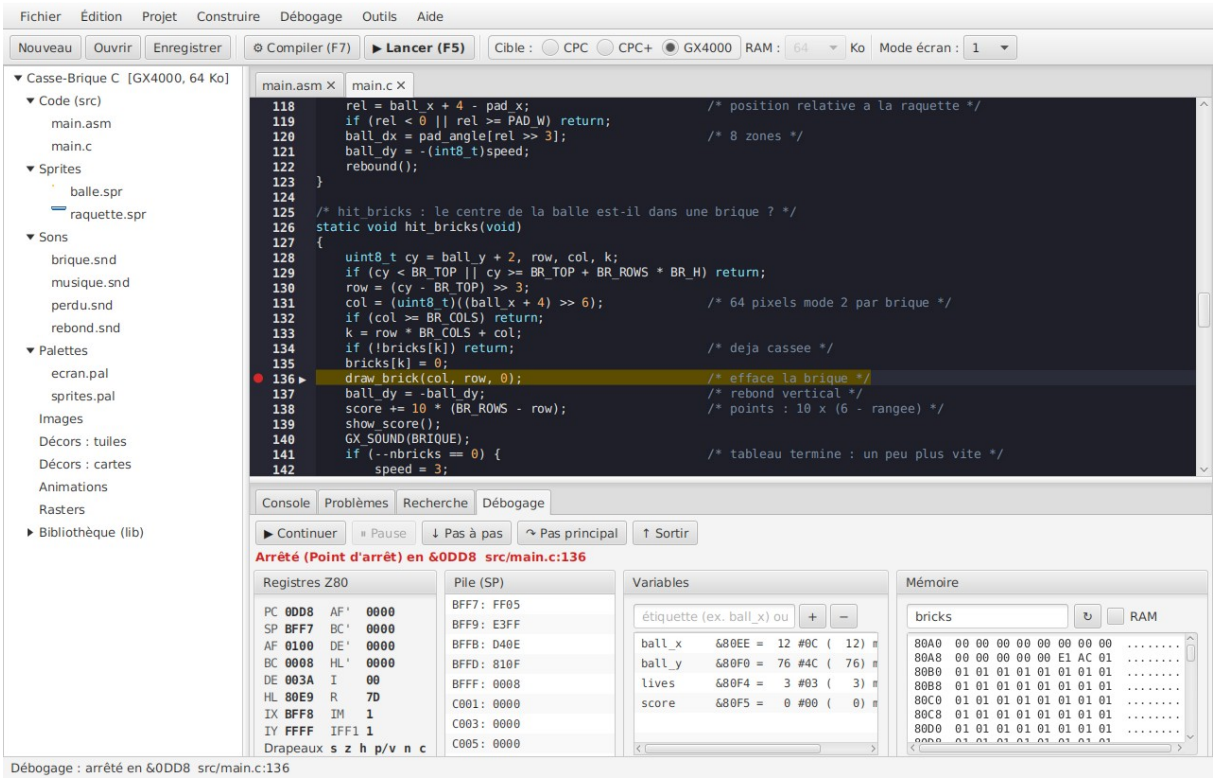
```
static const uint8_t raster_bricks[] = {
    BR_TOP - 2,                                     /* un peu avant la brique */
    RASTER_INK_ENTRY(0x00F2, BR_TOP - 1 + 1 * BR_H),  /* rouge (&0GRB) */
    RASTER_INK_ENTRY(0x08F0, BR_TOP - 1 + 2 * BR_H),  /* orange */
    RASTER_INK_ENTRY(0x0FF0, BR_TOP - 1 + 3 * BR_H),  /* jaune */
    RASTER_INK_ENTRY(0x0F40, BR_TOP - 1 + 4 * BR_H),  /* vert */
    RASTER_INK_ENTRY(0x0F0F, BR_TOP - 1 + 5 * BR_H),  /* cyan */
    RASTER_INK_ENTRY(0x044F, BR_TOP - 1 + 6 * BR_H),  /* bleu */
    RASTER_INK_ENTRY(0x0FFF, 199),                  /* retour au blanc (score) */
    RASTER_END_ENTRY                                /* fin d'image, armement */
};
```

12.8 Compiler et déboguer le C

F5 compile et lance, comme pour l'assembleur. Le studio compile chaque fichier .c avec SDCC, traduit le résultat pour rasm, puis rasm assemble le tout. Les erreurs du C s'affichent dans **Problèmes** (double-clic : la ligne du .c) :

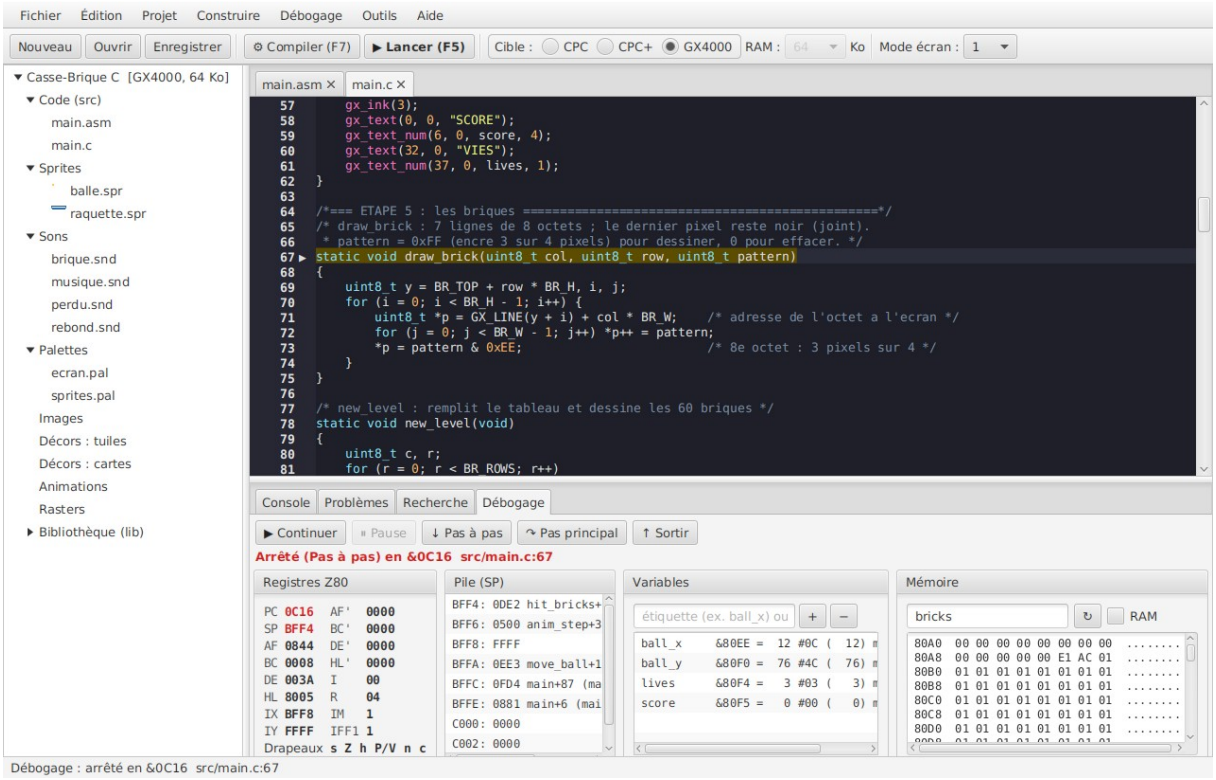
```
== C (SDCC)
  src/main.c:151: error 20: Undefined identifier 'lifes'
== ECHEC (C)
```

Le débogueur fonctionne sur les lignes C. Clic dans la marge (ou F9) sur la ligne `draw_brick(col, row, 0);` de hit_bricks, F5, puis tir pour servir : à la première brique touchée, le jeu s'arrête sur la ligne C.



Arrêt sur une ligne C ; dans Variables, les noms C (ball_x, lives…), dans Mémoire le tableau bricks

- **F11** avance d'une **ligne C** ; sur un appel, il entre dans la fonction (ici draw_brick).
- **F10** exécute la ligne entière, appels compris ; **Maj+F11** termine la fonction ; **F8** relance le jeu.
- Dans **Variables**, taper simplement le nom C (`ball_y`, `score`), même pour une variable `static`.
- La pile montre l'enchaînement des fonctions C : draw_brick appelée par hit_bricks, appelée par move_ball, appelée par main.



F11 : entrée dans draw_brick

Les définitions de **Projet > Propriétés** sont aussi passées au compilateur C : `BR_ROWS=2` fonctionne comme en assembleur grâce au `#ifndef BR_ROWS` de l'étape 1.

12.9 Ajouter de l'assembleur

Le C et l'assembleur se mélangent librement. Pour accélérer une routine, l'écrire dans un fichier src/outils.asm, l'inclure dans main.asm (`include "outils.asm"`, après `include "../lib/c/gx_c.asm"`) et la déclarer dans main.c. Le nom en assembleur prend un _ devant ; les paramètres sont sur la pile, le résultat revient dans L (8 bits) ou HL (16 bits) ; IX doit être conservé.

```
/* main.c */
uint8_t add3(uint8_t a, uint8_t b, uint8_t c) __sdcccall(0);

; src/outils.asm
_add3:  ld hl,2
        add hl,sp               ; HL -> premier parametre
        ld a,(hl)
        inc hl
        add a,(hl)
        inc hl
        add a,(hl)
        ld l,a                  ; resultat dans L
        ret
```

Inversement, une variable globale C `niveau` se lit en assembleur sous le nom `_niveau`, et une fonction C `f` s'appelle par `call _f`. Exercice : réécrire draw_brick en assembleur en reprenant celle du chapitre 5, et comparer la vitesse en effaçant 60 briques.



Le casse-brique en C : même jeu, même cartouche GX4000